

รายงานการผลทดลองปฏิบัติการ

1/2569 240-391 & 241-251 Robot Control Module

สมาชิก

นาย รัชชานนท์ ฉิมใหญ่ 6610110619

นาย พลกฤต บัวลอย 6810110223

นาย พัทธวัฒน์ พลเพชร 6810110230

นาย ปิณณ์ สุวรรณแสง 6810110638

วันที่ลงปฏิบัติการ 17/9/2569

หัวข้อการปฏิบัติการ Lab การควบคุม Gimbal ยิงป้ายสี

สาขาวิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์

มหาวิทยาลัยสงขลานครินทร์

1. วัตถุประสงค์

- เพื่อศึกษาและประยุกต์ใช้ระบบประมวลผลภาพ (Computer Vision) บนโมเดลสี HSV (Hue-Saturation-Value) ในการตรวจจับและแยกแยะสีของเป้าหมาย (Red, Green, Blue, Dark Yellow)
- เพื่อศึกษาสูตรการคำนวณและออกแบบป้อนกลับควบคุมความเร็วการหมุน Gimbal (Yaw/Pitch Tracking Controller) เพื่อติดตามเป้าหมายสีให้อยู่กึ่งกลางภาพอัตโนมัติ

2. โครงสร้างโปรแกรม

```
import cv2
import numpy as np
import tkinter as tk
from tkinter import ttk, scrolledtext
import threading
import time
import csv
from datetime import datetime

# Matplotlib Setup
import matplotlib
matplotlib.use("TkAgg")
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

try:
    from robomaster import robot, blaster
    ROBOMASTER_AVAILABLE = True
except ImportError:
    ROBOMASTER_AVAILABLE = False
    print("Warning: ไม่พบ robomaster SDK (รันในโหมดจำลองด้วย Webcam)")

COLOR_RANGES = {
    'Red': [
        (np.array([0, 120, 70]), np.array([10, 255, 255])),
        (np.array([165, 120, 70]), np.array([180, 255, 255]))
    ],
    'Green': [
        (np.array([35, 50, 35]), np.array([95, 255, 255]))
    ],
    'Blue': [
        (np.array([90, 45, 15]), np.array([135, 255, 255]))
    ]
}
```

```

    ],
    'Dark Yellow': [
        (np.array([18, 170, 120]), np.array([30, 255, 255]))
    ]
}

class RoboMasterTargetApp:
    def __init__(self, window):
        self.window = window
        self.window.title("RoboMaster Auto Target & Telemetry Tracking")
        self.window.geometry("1150x700")

        self.fire_mode = tk.StringVar(value="IR")
        self.target_count_var = tk.IntVar(value=3)
        self.color_sequence = ["Red", "Blue", "Green", "Dark Yellow"]
        self.is_running = False
        self.current_target_idx = 0
        self.last_log_time = 0

        self.is_cooldown = False
        self.cooldown_end_time = 0
        self.lock_count = 0
        self.is_firing_in_progress = False

        # โครงสร้างค่านวณตำแหน่งองศาจริง Gimbal Angle
        self.gimbal_yaw_deg = 0.0
        self.gimbal_pitch_deg = 0.0
        self.last_frame_time = time.time()

        self.csv_file = None
        self.csv_writer = None
        self.start_time = 0
        self.time_history = []
        self.yaw_angle_history = []
        self.pitch_angle_history = []
        self.fire_events = []
        self.target_yaw_levels = {}

        self.setup_gui()

    if ROBOMASTER_AVAILABLE:
        self.ep_robot = robot.Robot()
        self.ep_robot.initialize(conn_type="ap")
        self.ep_gimbal = self.ep_robot.gimbal
        self.ep_blaster = self.ep_robot.blaster
        self.ep_camera = self.ep_robot.camera
    else:
        self.ep_robot = None

```

```

def setup_gui(self):
    left_frame = ttk.Frame(self.window)
    left_frame.pack(side="left", fill="both", expand=False, padx=10, pady=5)

    mode_frame = ttk.LabelFrame(left_frame, text=" 1. เลือกโหมดการยิง")
    mode_frame.pack(fill="x", pady=5)
    ttk.Radiobutton(mode_frame, text="ยิงอินฟราเรด (IR)", variable=self.fire_mode,
value="IR").pack(side="left", padx=15, pady=5)
    ttk.Radiobutton(mode_frame, text="ยิงกระสุนเจล (Gel)", variable=self.fire_mode,
value="Gel").pack(side="left", padx=15, pady=5)

    count_frame = ttk.LabelFrame(left_frame, text=" 2. เลือกจำนวนป้ายที่จะยิง")
    count_frame.pack(fill="x", pady=5)
    ttk.Label(count_frame, text="จำนวนป้าย:").pack(side="left", padx=10, pady=5)
    count_combo = ttk.Combobox(count_frame, values=[1, 2, 3, 4],
textvariable=self.target_count_var, state="readonly", width=8)
    count_combo.pack(side="left", padx=5, pady=5)
    count_combo.bind("<<ComboboxSelected>>", self.update_sequence_ui)

    self.seq_frame = ttk.LabelFrame(left_frame, text=" 3. เลือกลำดับสีที่จะยิง")
    self.seq_frame.pack(fill="x", pady=5)

    self.seq_combos = []
    colors = ["Red", "Green", "Blue", "Dark Yellow"]
    for i in range(4):
        f = ttk.Frame(self.seq_frame)
        f.pack(fill="x", padx=5, pady=2)
        ttk.Label(f, text=f"นัดที่ {i+1}:").pack(side="left", padx=5)
        combo = ttk.Combobox(f, values=colors, state="readonly", width=15)
        combo.set(self.color_sequence[i])
        combo.pack(side="left", padx=5)
        self.seq_combos.append(combo)

    self.update_sequence_ui()

    btn_frame = ttk.Frame(left_frame)
    btn_frame.pack(fill="x", pady=5)

    self.start_btn = ttk.Button(btn_frame, text="เริ่มระบบ (Start)",
command=self.start_processing)
    self.start_btn.pack(side="left", padx=5, expand=True, fill="x")

    self.stop_btn = ttk.Button(btn_frame, text="หยุด (Stop)", command=self.stop_processing,
state="disabled")
    self.stop_btn.pack(side="right", padx=5, expand=True, fill="x")

```

```

        self.status_label = ttk.Label(left_frame, text="สถานะ: พร้อมทำงาน", font=("Arial", 10,
"bold"))
        self.status_label.pack(pady=3)

        log_frame = ttk.LabelFrame(left_frame, text=" Log การตรวจจับค่าสี ")
        log_frame.pack(fill="both", expand=True, pady=5)

        self.log_box = scrolledtext.ScrolledText(log_frame, height=12, width=42,
state='disabled', font=("Consolas", 8))
        self.log_box.pack(fill="both", expand=True, padx=5, pady=5)

        right_frame = ttk.LabelFrame(self.window, text=" Gimbal Time Response & Tracking
Analysis ")
        right_frame.pack(side="right", fill="both", expand=True, padx=10, pady=5)

        self.fig, (self.ax_yaw, self.ax_pitch) = plt.subplots(2, 1, sharex=True, figsize=(6,
5), dpi=95)
        self.fig.suptitle("Gimbal Time Response & IR Fire Tracking", fontsize=12,
fontweight='bold')
        self.fig.tight_layout(rect=[0, 0.03, 1, 0.95])

        self.canvas = FigureCanvasTkAgg(self.fig, master=right_frame)
        self.canvas.get_tk_widget().pack(fill="both", expand=True, padx=5, pady=5)

def update_sequence_ui(self, event=None):
    count = self.target_count_var.get()
    for i in range(4):
        if i < count:
            self.seq_combos[i].config(state="readonly")
        else:
            self.seq_combos[i].config(state="disabled")

def log(self, text):
    timestamp = time.strftime("%H:%M:%S")
    msg = f"[{timestamp}] {text}\n"
    print(msg, end="")
    if hasattr(self, 'log_box'):
        try:
            self.log_box.config(state='normal')
            self.log_box.insert(tk.END, msg)
            self.log_box.see(tk.END)
            self.log_box.config(state='disabled')
        except:
            pass

def init_csv_logger(self):
    filename = f"telemetry_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv"
    self.csv_file = open(filename, mode='w', newline='', encoding='utf-8')

```

```

self.csv_writer = csv.writer(self.csv_file)
self.csv_writer.writerow([
    "Timestamp", "Elapsed_Time_Sec", "Target_Color",
    "Yaw_Angle_Deg", "Pitch_Angle_Deg", "Yaw_Speed", "Pitch_Speed",
    "Is_Locked", "Fire_Event", "Fire_Mode"
])
self.log(f"เริ่มบันทึก Telemetry ไฟล์: {filename}")

def log_telemetry_data(self, elapsed_time, target_color, yaw_deg, pitch_deg, yaw_speed,
pitch_speed, is_locked, is_fire):
    if self.csv_writer:
        timestamp_str = datetime.now().strftime('%H:%M:%S.%f')[:-3]
        self.csv_writer.writerow([
            timestamp_str, round(elapsed_time, 3), target_color,
            round(yaw_deg, 2), round(pitch_deg, 2), round(yaw_speed, 2),
            round(pitch_speed, 2),
            is_locked, 1 if is_fire else 0, self.fire_mode.get()
        ])
        self.csv_file.flush()

    self.time_history.append(elapsed_time)
    self.yaw_angle_history.append(yaw_deg)
    self.pitch_angle_history.append(pitch_deg)

def update_plot_loop(self):
    if self.is_running and len(self.time_history) > 0:
        self.ax_yaw.clear()
        self.ax_pitch.clear()

        # 1. กราฟ Gimbal Yaw Angle (°)
        self.ax_yaw.plot(self.time_history, self.yaw_angle_history, label='Gimbal Yaw
Angle (°)', color='#1f77b4', linewidth=1.5)

        line_styles = ['--', '-.', ':']
        line_colors = ['red', 'gray', 'green', 'orange']
        for i, (color_name, angle_val) in enumerate(self.target_yaw_levels.items()):
            c = line_colors[i % len(line_colors)]
            ls = line_styles[i % len(line_styles)]
            self.ax_yaw.axhline(angle_val, color=c, linestyle=ls, alpha=0.7,
label=f'{color_name} ({angle_val:.1f}°)')

        # 2. กราฟ Gimbal Pitch Angle (°)
        self.ax_pitch.plot(self.time_history, self.pitch_angle_history, label='Gimbal
Pitch Angle (°)', color='#ff7f0e', linewidth=1.5)

        # 3. มาร์กเกอร์ดาวแดงแสดงตำแหน่งยิง
        has_fire_legend = False
        for evt in self.fire_events:

```

```

        t_evt, yaw_evt, pitch_evt, count_evt, color_evt = evt

        label_star = 'IR Fire Event' if not has_fire_legend else ""
        self.ax_yaw.plot(t_evt, yaw_evt, '*', color='red', markersize=11,
label=label_star)
        self.ax_pitch.plot(t_evt, pitch_evt, '*', color='red', markersize=9)
        has_fire_legend = True

        offset_y = 6 if yaw_evt < 0 else -10
        self.ax_yaw.annotate(
            f"Fire!\n{count_evt}. {color_evt}\n({yaw_evt:.1f}°)",
            xy=(t_evt, yaw_evt),
            xytext=(t_evt, yaw_evt + offset_y),
            arrowprops=dict(facecolor='black', arrowstyle='->', lw=1.0),
            ha='center', va='center', fontsize=6.5, color='darkred',
fontweight='bold'
        )

        self.ax_yaw.set_ylabel("Yaw Angle (°)", fontsize=9)
        self.ax_yaw.grid(True, linestyle=':', alpha=0.6)
        self.ax_yaw.legend(loc='upper right', fontsize=7)

        self.ax_pitch.set_ylabel("Pitch Angle (°)", fontsize=9)
        self.ax_pitch.set_xlabel("Time (s)", fontsize=9)
        self.ax_pitch.grid(True, linestyle=':', alpha=0.6)
        self.ax_pitch.legend(loc='upper right', fontsize=7)

        self.canvas.draw_idle()

        if self.is_running:
            self.window.after(250, self.update_plot_loop)

    def mask_gun_barrel(self, frame):
        h, w, _ = frame.shape
        clean_frame = frame.copy()
        cv2.rectangle(clean_frame, (int(w * 0.28), int(h * 0.65)), (int(w * 0.72), h), (0,
0, 0), -1)
        return clean_frame

    def get_color_mask(self, hsv_frame, color_name):
        ranges = COLOR_RANGES[color_name]
        mask = np.zeros(hsv_frame.shape[:2], dtype=np.uint8)
        for (lower, upper) in ranges:
            mask |= cv2.inRange(hsv_frame, lower, upper)
        return mask

    def fire_weapon_task(self):
        if self.is_firing_in_progress:

```

```

        return
    self.is_firing_in_progress = True
    try:
        mode = self.fire_mode.get()
        if ROBOMASTER_AVAILABLE and self.ep_blaster:
            if mode == "IR":
                self.ep_blaster.fire(fire_type=blaster.INFRARED_FIRE, times=1)
            else:
                self.ep_blaster.fire(fire_type=blaster.WATER_FIRE, times=1)
        self.log(f"-> [FIRE!] กำลังยิงด้วยโหมด: {mode}")
    except Exception as e:
        self.log(f"Fire Error: {e}")
    finally:
        self.is_firing_in_progress = False

def draw_crosshair(self, frame, center_x, center_y):
    color = (0, 255, 255)
    size = 15
    thickness = 2
    cv2.line(frame, (center_x - size, center_y), (center_x + size, center_y), color,
thickness)
    cv2.line(frame, (center_x, center_y - size), (center_x, center_y + size), color,
thickness)
    cv2.circle(frame, (center_x, center_y), 4, (0, 0, 255), -1)

def create_combined_view(self, img1, img2, side_width=960, side_height=540):
    res_1 = cv2.resize(img1, (side_width, side_height))
    res_2 = cv2.resize(img2, (side_width, side_height))
    return np.hstack((res_1, res_2))

def process_loop(self):
    target_count = self.target_count_var.get()
    self.color_sequence = [self.seq_combos[i].get() for i in range(target_count)]
    self.current_target_idx = 0
    self.is_cooldown = False
    self.lock_count = 0

    # รีเซ็ตตำแหน่งมุม Gimbal เริ่มต้นที่ศูนย์กลาง (0°, 0°)
    self.gimbal_yaw_deg = 0.0
    self.gimbal_pitch_deg = 0.0
    self.last_frame_time = time.time()

    self.time_history.clear()
    self.yaw_angle_history.clear()
    self.pitch_angle_history.clear()
    self.fire_events.clear()
    self.target_yaw_levels.clear()
    self.start_time = time.time()

```

```

self.init_csv_logger()

if ROBOMASTER_AVAILABLE:
    self.log("กำลังรีเซ็ต Gimbal กลับจุดศูนย์ก่กลาง...")
    self.status_label.config(text="กำลังรีเซ็ต Gimbal...")
    self.ep_gimbal.recenter().wait_for_completed()
    self.ep_camera.start_video_stream(display=False)

cap = None if ROBOMASTER_AVAILABLE else cv2.VideoCapture(0)

scan_direction = 1
scan_counter = 0

self.log(f"เริ่มการทำงาน ถึง {target_count} เป้า | ลำดับสี: {'', '.join(self.color_sequence)}")
self.window.after(200, self.update_plot_loop)

while self.is_running and self.current_target_idx < len(self.color_sequence):
    now = time.time()
    dt = now - self.last_frame_time
    self.last_frame_time = now

    target_color = self.color_sequence[self.current_target_idx]
    self.status_label.config(text=f"กำลังเล็ง: {target_color} (นัดที่ {self.current_target_idx
+ 1}/{target_count})")

    frame = self.ep_camera.read_cv2_image(strategy="newest") if ROBOMASTER_AVAILABLE
else cap.read()[1]
    if frame is None:
        continue

    h, w, _ = frame.shape
    center_x, center_y = w // 2, h // 2

    self.draw_crosshair(frame, center_x, center_y)

    masked_frame = self.mask_gun_barrel(frame)
    hsv = cv2.cvtColor(masked_frame, cv2.COLOR_BGR2HSV)
    color_mask = self.get_color_mask(hsv, target_color)

    kernel = np.ones((5, 5), np.uint8)
    color_mask = cv2.morphologyEx(color_mask, cv2.MORPH_OPEN, kernel)
    color_mask = cv2.morphologyEx(color_mask, cv2.MORPH_CLOSE, kernel)

    mask_bgr = cv2.cvtColor(color_mask, cv2.COLOR_GRAY2BGR)

    if self.is_cooldown:
        if time.time() >= self.cooldown_end_time:

```

```

        self.is_cooldown = False
        self.lock_count = 0
        if ROBOMASTER_AVAILABLE:
            for _ in range(3):
                self.ep_camera.read_cv2_image(strategy="newest")
        else:
            cv2.putText(frame, f"Reloading... Next Target: {target_color}", (10,
30),

                        cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)
            cv2.putText(mask_bgr, f"HSV Mask: {target_color}", (10, 30),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2)

            combined_view = self.create_combined_view(frame, mask_bgr)
            cv2.imshow("RoboMaster Camera & Debug View", combined_view)
            cv2.waitKey(1)
            continue

        contours, _ = cv2.findContours(color_mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
        target_found = False

        valid_contours = []
        for c in contours:
            area = cv2.contourArea(c)
            if 40 < area < (w * h * 0.25):
                x, y, bw, bh = cv2.boundingRect(c)
                target_cy = y + bh // 2
                if target_cy > int(h * 0.25):
                    valid_contours.append((c, area))

        if valid_contours:
            best_contour, best_area = max(valid_contours, key=lambda item: item[1])
            target_found = True

            x, y, bw, bh = cv2.boundingRect(best_contour)
            target_cx, target_cy = x + bw // 2, y + bh // 2

            hsv_val = hsv[target_cy, target_cx]
            err_x = target_cx - center_x
            err_y = target_cy - center_y

            # จำนวนความเร็วควบคุม Gimbal
            if abs(err_x) > 15:
                yaw_speed = float(np.sign(err_x) * max(3.0, min(14.0, abs(err_x) *
0.08)))
            else:
                yaw_speed = 0.0

```

```

        if abs(err_y) > 15:
            pitch_speed = float(-np.sign(err_y) * max(3.0, min(10.0, abs(err_y) *
0.08)))
        else:
            pitch_speed = 0.0

        # คำนวณสะสมองศาการหมุนจริงของ Gimbal (Integrating Speed over dt)
        self.gimbal_yaw_deg += yaw_speed * dt * 1.5
        self.gimbal_pitch_deg += pitch_speed * dt * 1.5

        elapsed = time.time() - self.start_time
        is_locked_flag = abs(err_x) <= 15 and abs(err_y) <= 15

        # บันทึกมุมหมุนจริงลง Telemetry
        self.log_telemetry_data(elapsed, target_color, self.gimbal_yaw_deg,
self.gimbal_pitch_deg, yaw_speed, pitch_speed, is_locked_flag, False)

        if time.time() - self.last_log_time > 0.5:
            self.log(f"พบ[{target_color}] | HSV: H={hsv_val[0]} S={hsv_val[1]}
V={hsv_val[2]} | Gimbal Angle: Yaw={self.gimbal_yaw_deg:.1f}°,
Pitch={self.gimbal_pitch_deg:.1f}°")
            self.last_log_time = time.time()

        cv2.rectangle(frame, (x, y), (x + bw, y + bh), (0, 255, 0), 2)
        cv2.circle(frame, (target_cx, target_cy), 4, (0, 0, 255), -1)

        log_str = f"HSV:[{hsv_val[0]},{hsv_val[1]},{hsv_val[2]}]
Yaw:{self.gimbal_yaw_deg:.1f}deg"
        cv2.putText(frame, log_str, (x, max(15, y - 8)), cv2.FONT_HERSHEY_SIMPLEX,
0.45, (0, 255, 0), 1)

        if ROBOMASTER_AVAILABLE:
            self.ep_gimbal.drive_speed(pitch_speed=pitch_speed, yaw_speed=yaw_speed)

        if is_locked_flag:
            self.lock_count += 1
            if self.lock_count >= 3:
                if ROBOMASTER_AVAILABLE:
                    self.ep_gimbal.drive_speed(pitch_speed=0, yaw_speed=0)

            self.log(f"ล็อกเป้าหมายสี {target_color} สำเร็จ! (นัดที่ {self.current_target_idx +
1}/{target_count})")

        # บันทึกระดับองศาอ้างอิงและจุดยิง
        self.target_yaw_levels[f"Target {self.current_target_idx + 1}
({target_color})"] = self.gimbal_yaw_deg
        self.fire_events.append((elapsed, self.gimbal_yaw_deg,
self.gimbal_pitch_deg, self.current_target_idx + 1, target_color))

```

```

        self.log_telemetry_data(elapsed, target_color, self.gimbal_yaw_deg,
self.gimbal_pitch_deg, 0, 0, True, True)

        threading.Thread(target=self.fire_weapon_task, daemon=True).start()
        self.current_target_idx += 1
        self.is_cooldown = True
        self.cooldown_end_time = time.time() + 1.8
        self.lock_count = 0
    else:
        self.lock_count = max(0, self.lock_count - 1)

if not target_found and not self.is_cooldown:
    self.lock_count = 0
    scan_counter += 1
    if scan_counter > 35:
        scan_direction *= -1
        scan_counter = 0

    yaw_speed = float(8 * scan_direction)
    self.gimbal_yaw_deg += yaw_speed * dt * 1.5

    elapsed = time.time() - self.start_time
    self.log_telemetry_data(elapsed, target_color, self.gimbal_yaw_deg,
self.gimbal_pitch_deg, yaw_speed, 0, False, False)

    if time.time() - self.last_log_time > 1.5:
        self.log(f"ค้นหาเป้าหมาย [{target_color}]...")
        self.last_log_time = time.time()

    if ROBOMASTER_AVAILABLE:
        self.ep_gimbal.drive_speed(pitch_speed=0, yaw_speed=yaw_speed)

    cv2.putText(frame, f"Target ({min(self.current_target_idx+1,
target_count)}/{target_count}): {target_color}", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255), 2)
    cv2.putText(mask_bgr, f"HSV Mask: {target_color}", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2)

    combined_view = self.create_combined_view(frame, mask_bgr)
    cv2.imshow("RoboMaster Camera & Debug View", combined_view)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

while self.is_cooldown and self.is_running:
    time.sleep(0.1)

```

```

if self.current_target_idx >= target_count and self.is_running:
    self.log(f"อิงครบทั้ง {target_count} สีเรียบร้อยแล้ว! กำลังรอ 5 วินาทีก่อนปิดระบบ...")
    end_time = time.time() + 5.0

    while self.is_running and time.time() < end_time:
        remaining_sec = int(np.ceil(end_time - time.time()))
        self.status_label.config(text=f"สถานะ: อิงครบแล้ว (ปิดใน {remaining_sec}s)")

        frame = self.ep_camera.read_cv2_image(strategy="newest") if
ROBOMASTER_AVAILABLE else cap.read()[1]
        if frame is not None:
            h, w, _ = frame.shape
            self.draw_crosshair(frame, w // 2, h // 2)
            cv2.putText(frame, f"Completed! Closing in {remaining_sec}s...", (10,
30),

                                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)

            blank_mask = np.zeros_like(frame)
            combined_view = self.create_combined_view(frame, blank_mask)
            cv2.imshow("RoboMaster Camera & Debug View", combined_view)
            if cv2.waitKey(1) & 0xFF == ord('q'):
                break
            time.sleep(0.03)

    if cap:
        cap.release()
    cv2.destroyAllWindows()

    self.stop_processing()

def start_processing(self):
    if self.is_running:
        return
    self.is_running = True
    self.start_btn.config(state="disabled")
    self.stop_btn.config(state="normal")
    threading.Thread(target=self.process_loop, daemon=True).start()

def stop_processing(self):
    self.is_running = False
    self.start_btn.config(state="normal")
    self.stop_btn.config(state="disabled")

    if self.csv_file:
        try:
            self.csv_file.close()
            self.log("ปิดไฟล์บันทึก CSV Telemetry เรียบร้อยแล้ว")
        except:

```

```

        pass
        self.csv_file = None

    if ROBOMASTER_AVAILABLE:
        if self.ep_gimbal:
            try:
                self.ep_gimbal.drive_speed(pitch_speed=0, yaw_speed=0)
            except:
                pass
        if self.ep_camera:
            try:
                self.ep_camera.stop_video_stream()
            except:
                pass

    self.status_label.config(text="สถานะ: หยุดการทำงาน")
    self.log("หยุดการทำงานของระบบ")

    def on_close(self):
        self.stop_processing()
        if ROBOMASTER_AVAILABLE and self.ep_robot:
            self.ep_robot.close()
        self.window.destroy()

if __name__ == "__main__":
    root = tk.Tk()
    app = RoboMasterTargetApp(root)
    root.protocol("WM_DELETE_WINDOW", app.on_close)
    root.mainloop()

```

3. คำอธิบายคำสั่งสำคัญ

Function	หน้าที่
COLOR_RANGES	พจนานุกรมเก็บขอบเขตค่า HSV (Lower/Upper) สำหรับกรองสี Red, Green, Blue และ Dark Yellow
setup_gui()	สร้างหน้าต่างอินเทอร์เฟซด้วย Tkinter สำหรับตั้งค่าโหมดการยิง, ลำดับสี, ปุ่มควบคุม และฝังพล็อตกราฟ Matplotlib
mask_gun_barrel()	สร้าง Mask สีเหลี่ยมสีดำบดบังบริเวณลำกล้องปืนส่วนล่างเฟรมภาพ ป้องกันสัญญาณรบกวนสีจากโครงรถ
get_color_mask()	แปลงภาพ BGR เป็น HSV และใช้ cv2.inRange() กรองเฉพาะพิกเซลของเป้าหมายสีที่กำลังติดตาม

fire_weapon_task()	สั่งการ Blaster ยิงกระสุน IR หรือ Gel ใน Background Thread เพื่อไม่ให้กระทบการประมวลผลกล้องวิดีโอหลัก
init_csv_logger() / log_telemetry_data()	สร้างไฟล์ CSV บันทึกค่า Telemetry (Timestamp, Yaw/Pitch Angle, Yaw/Pitch Speed, Is_Locked, Fire_Event)
update_plot_loop()	อัปเดตการแสดงผลกราฟ Time Response บน GUI แบบ Real-time ทุกๆ 250 ms
process_loop()	ลูปหลักควบคุมระบบ (Main Loop): กวาดสแกนหาเป้า, คำนวณ Error, ปรับความเร็ว Gimbal, ล็อกเป้าหมาย และยิงกระสุนตามลำดับ
COLOR_RANGES	พจนานุกรมเก็บขอบเขตค่า HSV (Lower/Upper) สำหรับกรองสี Red, Green, Blue และ Dark Yellow
setup_gui()	สร้างหน้าต่างอินเทอร์เฟซด้วย Tkinter สำหรับตั้งค่าโหมดการยิง, ลำดับสี, ปุ่มควบคุม และฝังพล็อตกราฟ Matplotlib
mask_gun_barrel()	สร้าง Mask สีเหลี่ยมสีดำบดบังบริเวณลำกล้องปืนส่วนล่างเฟรมภาพ ป้องกันสัญญาณรบกวนจากโครงรถ

4. การทดลองและขั้นตอนการดำเนินงาน

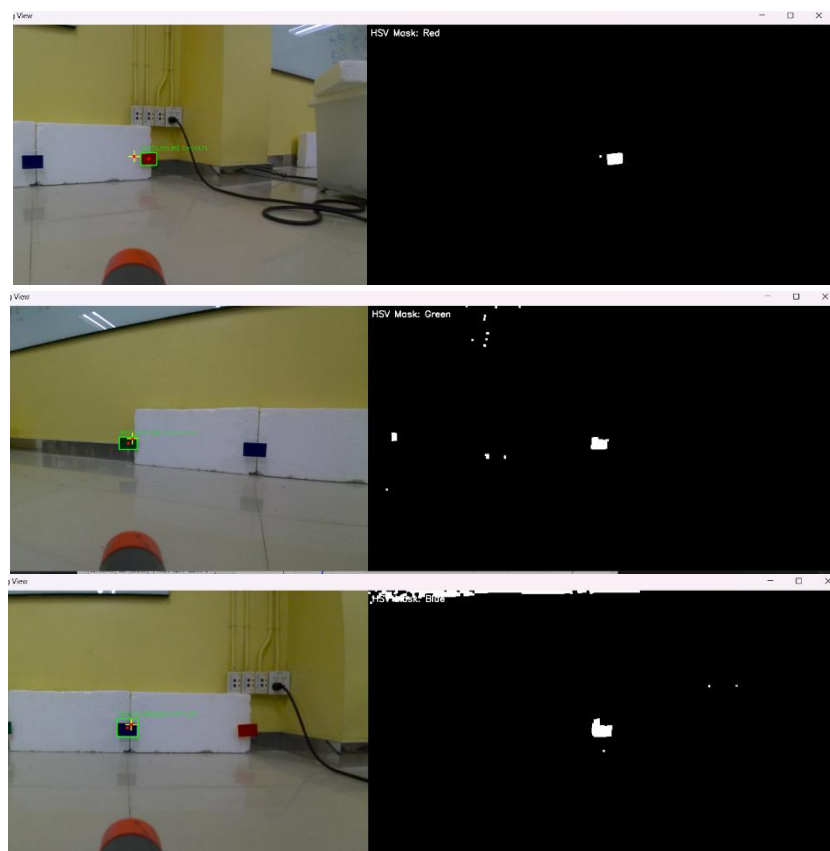
4.1 สภาพแวดล้อมและช่วงค่าสี HSV (COLOR_RANGES)

จัดตั้งเป้าหมายสีรูปทรงต่างๆ ไว้ตรงหน้าหุ่นยนต์ระยะ 1.0 - 1.5 เมตร โดยกำหนดช่วงค่าสี HSV ดังตาราง:

ชื่อเป้าสี (Target Color)	ช่วงค่า HSV Min (Lower)	ช่วงค่า HSV Max (Upper)
Red (แดง) - ช่วงที่ 1	[0, 120, 70]	[10, 255, 255]
Red (แดง) - ช่วงที่ 2	[165, 120, 70]	[180, 255, 255]
Green (เขียว)	[35, 50, 35]	[95, 255, 255]
Blue (น้ำเงิน)	[90, 45, 15]	[135, 255, 255]
Dark Yellow (เหลืองเข้ม)	[18, 170, 120]	[30, 255, 255]

4.2 ขั้นตอนการทำงานของโปรแกรม

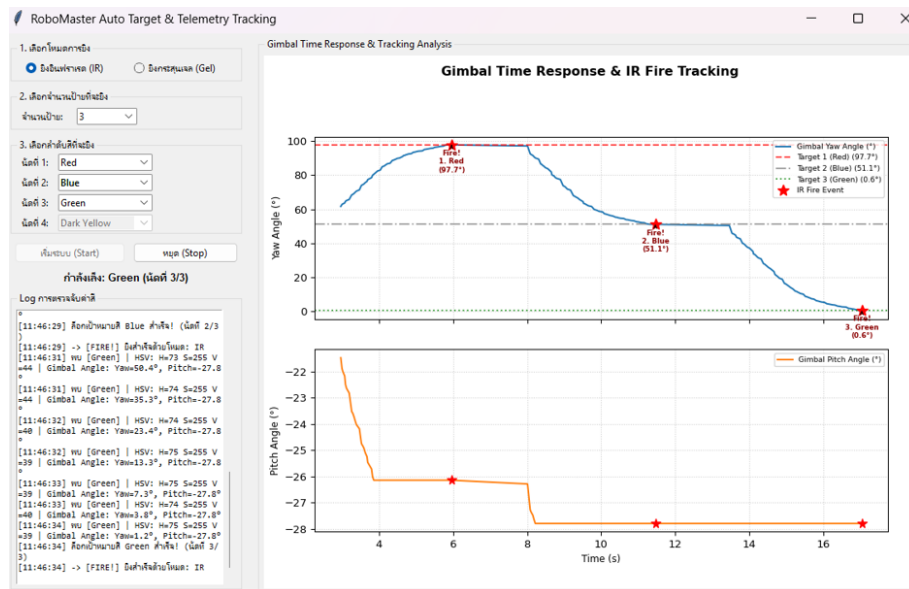
- **เริ่มต้นระบบและ GUI:** เลือกโหมดการยิง , กำหนดจำนวนป้าย (1-4 ป้าย), เลือกลำดับสี Red - > Blue -> Green แล้วกดปุ่ม "เริ่มระบบ (Start)"
- **รีเซ็ตตำแหน่ง Gimbal:** สั่ง recenter() ให้ Gimbal หันตรง (0° , 0°) และเปิดไฟล์ CSV บันทึกข้อมูล Telemetry
- **สแกนหาเป้าหมาย (Scan Sweep Mode):** หากยังไม่พบเป้าสียปัจจุบัน Gimbal จะกวาดซ้าย-ขวาด้วยความเร็ว ± 8 deg/s เพื่อค้นหา
- **การติดตามเป้าหมาย (Tracking Mode):** เมื่อพบเป้าหมายสี โปรแกรมจะคำนวณ Error_X, Error_Y และสั่งการความเร็ว Gimbal ดึงเป้าหมายเข้าหาถึงกลางภาพ
- **การล็อกเป้าและยิง (Lock & Fire):** เมื่อจุดกึ่งกลางเป้าหมายอยู่ในรัศมี ≤ 15 px ติดต่อกัน 3 เฟรม ระบบจะหยุด Gimbal บันทึกข้อมูลตำแหน่ง และสั่ง Blaster ยิง IR ทันที
- **การหน่วงเวลาบรรจุ (Cooldown Period):** หน่วงเวลา 1.8 วินาที เพื่อรีโหลดและป้องกันการยิงซ้ำ ก่อนเปลี่ยนไปค้นหาและยิงเป้าหมายสีถัดไปจนครบภารกิจ



5. ผลการทดลองและวิเคราะห์ผล

5.1 รูปภาพแสดงกราฟตอบสนองทางเวลา (Gimbal Time Response & Fire Tracking)

ภาพกราฟ Time Response ด้านล่างแสดงการตอบสนองของมุม Yaw และ Pitch ของ Gimbal ขณะยิงเป้าหมาย



5.2 วิเคราะห์ผลการตอบสนองทางเวลา

- **ความแม่นยำในการตรวจจับสี:** การใช้ HSV Color Space ร่วมกับเทคนิค Morphological Operations และ Gun Barrel Masking ช่วยป้องกันสัญญาณรบกวนจากปากกระบอกปืนได้ ทำให้ระบบสามารถกรองเฉพาะเป้าหมายสี Red, Blue และ Green ได้อย่างแม่นยำ
- **สมรรถนะการติดตามเป้าหมาย:** อัลกอริทึม Proportional Tracking (P-Controller) สามารถปรับความเร็ว Gimbal ตามระยะ Error_X และ Error_Y ได้อย่างเรียบราบรื่น เมื่อเข้าใกล้ศูนย์กลางหน้าจอ (≤ 15 px) ความเร็วจะชะลอลงอย่างนุ่มนวล ช่วยป้องกันการ Overshoot และลดอาการสั่นไหว
- **เสถียรภาพในการล็อกและยิง:** เงื่อนไขล็อกเป้า 3 เฟรมติดกันร่วมกับ Cooldown 1.8 วินาที ช่วยให้ยิงอินฟราเรดเข้าเป้าหมายได้อย่างแม่นยำ และป้องกันปัญหาการล็อกสั่นไหวขณะสั่งยิง

ผลการทดลอง

การทดลองระบบควบคุม Gimbal ติดตามเป้าหมายสีด้วยระบบประมวลผลภาพ (HSV Color Space) บนหุ่นยนต์ DJI RoboMaster EP ประสบผลสำเร็จ โดยโปรแกรม สามารถควบคุมการกวาดสแกน ค้นหา ล็อกเป้าหมาย และยิงกระสุนใส่เป้าสี Red, Blue และ Green ได้ตรงตามลำดับอย่างแม่นยำทุกนัด โดยปราศจากอาการ Overshoot

ลิงก์วิดีโอ YouTube:

<https://www.youtube.com/watch?v=8RkeEF2v0fY>